

LLM có sinh test REST API tốt bằng người & EvoMaster?

So sánh **AI (LLM)** vs **test thủ công** vs **EvoMaster** trên các API đã cài lỗi sẵn — để đo được chính xác ai bắt được nhiều lỗi hơn.



Tóm tắt 30 giây (nếu chỉ nhớ 1 slide)

Toàn bộ nghiên cứu gói trong 3 câu.

- ① Tụi em cho **AI**, người, và EvoMaster cùng viết test cho 3 API thật.
- ② Tụi em **cài lỗi sẵn** vào code → biết chính xác có bao nhiêu lỗi → đo ai bắt được bao nhiêu.
- ③ Kết quả pilot: AI **phủ hết endpoint + nhiều kịch bản lỗi hơn người**, nhưng EvoMaster **bắt lỗi tính toán giỏi hơn**.

Đây là "thang máy pitch" — nếu thầy hỏi gấp "nghiên cứu của em là gì?", đọc đúng 3 câu này.

Dàn ý buổi bảo vệ

6 phần — đi theo mạch chuyện: vấn đề → khoảng trống → ý tưởng → cách làm → kết quả → bảo vệ.

① Vấn đề

Test API tốn; AI hứa hẹn — nhưng có đáng tin?

② Khoảng trống

59 paper, 3 lỗ hổng chưa ai lấp.

③ Ý tưởng & câu hỏi

3 đấu thủ + lỗi gieo sẵn → RQ1/2/3.

④ Cách làm (5 bước)

Step-by-step + ví dụ thật.

⑤ Kết quả pilot

Số liệu + cách đọc.

⑥ Bảo vệ

Threats + câu hỏi GV hay hỏi.

Vì sao test REST API lại quan trọng & tốn?

REST API = cách các phần mềm "nói chuyện" với nhau (app ↔ server).

Ở khắp nơi

Mọi app mobile/web đều gọi REST API. Một lỗi API = nhiều app hỏng theo.

Test rất tốn

Mỗi endpoint phải thử: input đúng, input sai, giá trị biên, lỗi... Viết tay rất lâu.

Ví von

Như đi kiểm tra **mọi cánh cửa** của một toà nhà: đúng khoá, sai khoá, cửa kẹt, cửa cấm...

→ Nếu có công cụ **tự sinh test** tốt, tiết kiệm cực nhiều công sức QA.

AI (LLM) làm gì trong bài này?

Cho AI đọc bản mô tả API (OpenAPI spec) → AI tự viết ra code test.

ĐẦU VÀO: OPENAPI SPEC

```
GET /api/bessj/{n}/{x}  
trả về: số (JSON)  
n: integer, x: number
```

AI (LLM) Suy luận

```
"n là số nguyên →  
thử n âm, n=0, n rất lớn,  
thử x sai kiểu..."
```

ĐẦU RA: TEST TỰ ĐỘNG

```
@Test bessj_n1()  
get("/api/bessj/1/2.0")  
.statusCode(anyOf(200,400))
```

AI **chỉ nhìn spec, không nhìn code nguồn** (black-box) — y như một tester thật ngoài đời.

CÂU HỎI CỐT LÕI

Câu hỏi không phải "AI sinh được test không?"

Mà là: "Test đó có tốt BẰNG người / EvoMaster không — trên những lỗi ta ĐẾM ĐƯỢC?"

Vì AI sinh test thì dễ; nhưng nếu test toàn dễ, bỏ sót lỗi thật → vô dụng mà ta không hề biết.

Tại em tìm khoảng trống thế nào?

Đọc có hệ thống (Systematic Literature Review) — không đoán mò.

1

Gom

Tìm các paper 2018–2026 về LLM & test REST API.

>

2

Lọc

Theo tiêu chí → còn **59 paper** chất lượng.

>

3

Lập bảng

Mỗi paper: tool, dataset, metric, kết quả.

>

4

Tìm lỗ hổng

Cái gì **chưa** ai làm đủ quan trọng?

→ Lộ ra **3 khoảng trống** lặp đi lặp lại ở cả những công trình mạnh.

Ba khoảng trống (lỗ hổng) tìm được

Mỗi cái kèm bằng chứng từ paper.

GAP-C · So sánh

Đúng 1 nghiên cứu so với **người** (EvoMaster)
— mà lại là tool cũ, **thua người 41% vs 82%**.

Chưa ai đặt AI vs người vs EvoMaster cùng
lúc. [proposal §2.3](#)

GAP-D · Dữ liệu

Lỗi chỉ được báo dạng **đếm 500-error** trên
API sống; không có bộ **lỗi gieo sẵn** → **không**
tính được Recall thật (bắt được mấy % lỗi).

GAP-M · Đo lường

Đánh giá thiên về coverage; **không** có thước
đo **kịch bản lỗi/biên** cho từng endpoint.

Nhóm tụi em **lấp cả 3** — đó là điểm mới.

Ý tưởng: một "trận đấu" công bằng

3 đấu thủ cùng viết test cho cùng API; trọng tài là bộ lỗi gieo sẵn.

AI • LLM

Đọc spec → sinh test tự động
đại diện "tự động hoá mới"

Người • Manual

Tester viết tay (EP/BVA)
chuẩn vàng truyền thống

EvoMaster

Tool tự động kỳ cựu (SOTA)
baseline mạnh nhất ngành

Trọng tài = **lỗi gieo sẵn**: ta biết chính xác có bao nhiêu lỗi, nên đo được ai bắt được bao nhiêu %.

Đây là điểm "không ai bắt bẻ được": có đáp án sẵn (ground truth) nên kết quả khách quan.

Ba câu hỏi nghiên cứu

Chốt trước, GV duyệt rồi KHÔNG đổi (tránh "chỉnh sau khi thấy số" — No-HARKing).

RQ1

Phủ endpoint

AI có chạm $\geq 90\%$ số endpoint? Loại nào hay bị bỏ?

$H0: \leq 90\%$ · $H1: > 90\%$

RQ2

Bắt lỗi gieo

AI bắt nhiều lỗi gieo hơn người & EvoMaster?
(Recall)

$H0: AI \leq \max(\text{người}, \text{Evo})$

RQ3

Kịch bản lỗi/biên

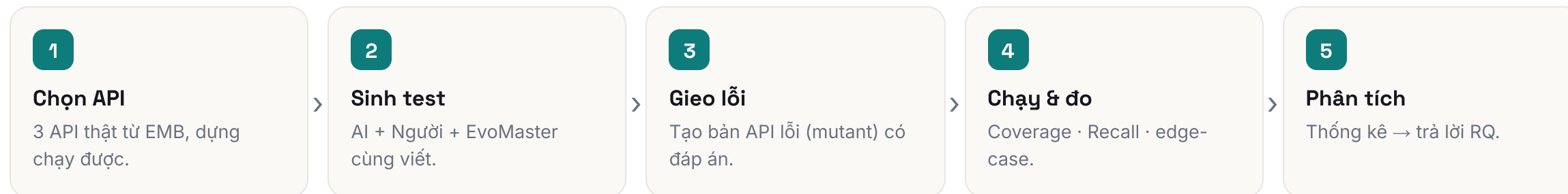
AI sinh nhiều kịch bản 4xx/5xx + biên per endpoint hơn người?

$H0: AI \leq \text{người}$

H0 = "không có gì khác biệt" (giả thuyết cần bác bỏ) · **H1** = điều tui em muốn chứng minh.

Cách làm — bức tranh lớn (5 bước)

Nhìn slide này là nắm toàn bộ quy trình; 4 slide sau zoom vào từng bước.



Tất cả **tự động + lặp lại được** (script + dữ liệu công khai EMB).

Bước 1 — Chọn API: bộ EMB

EMB = bộ "đề thi chuẩn" công khai của giới nghiên cứu (EvoMaster Benchmark).

API	LOẠI	ENDPOINT
rest-ncs	tính toán số	6
rest-scs	xử lý chuỗi	11
features-service	CRUD + DB thật	18
TỔNG	—	35

Vì sao EMB?

Công khai, nhiều paper dùng → uy tín, so sánh công bằng, không tự bịa data.

Vì sao 3 cái này?

ncs/scs nhỏ & ổn định (dễ kiểm soát); features là app thật có DB (thuyết phục).

Con số 35 endpoint khớp đúng tài liệu gốc EMB — không cãi được. [📄 table_emb.md](#)

Bước 2 — Ba đấu thủ sinh test thế nào?

Cùng 1 đầu vào (spec), không ai thấy lỗi gieo (blind).

AI (LLM)

Claude Sonnet 4.6 (pilot + full)
Đọc spec → sinh JUnit + RestAssured
prompt cố định, nhiệt độ 0

Người

Kỹ thuật EP + BVA
Chia lớp tương đương + thử giá trị biên
"chuẩn vàng" QA

EvoMaster

v6.0 black-box, 120 giây
Thuật toán tiến hoá tự dò
baseline mạnh

Blind protocol: cả 3 viết test **TRƯỚC** khi lỗi được gieo → không ai "học tủ" → công bằng.

 log AI: tool_uses=0

Bước 3 — ”Gieo lỗi” (mutation) là gì?

Cố ý sửa nhẹ code để tạo một con bug — rồi xem test nào phát hiện.

CODE GỐC (ĐÚNG)

```
if (n < 2)
  return 400;
```

MUTANT (ĐÃ GIEO LỖI)

```
if (n ≤ 2)
  return 400;
// đổi < thành ≤
```

HẬU QUẢ

```
n = 2:
gốc → 200 (đúng)
mutant → 400 (SAI!)
```

Test **"giết"** được mutant nếu: **pass** trên code gốc nhưng **fail** trên mutant.

Recall = số mutant bị giết / tổng mutant. Pilot: **133 mutant** (đáp án biết trước).

Bước 4 — Đo 3 thứ

Mỗi thước đo trả lời một câu hỏi khác nhau.

→ RQ1

Coverage

% endpoint được ít nhất 1 test chạm tới. "Có gọi tới chưa?"

→ RQ2

Recall lỗi gieo

% mutant bị giết. "Bắt được lỗi thật không?"

→ RQ3

Edge-case

Số kịch bản negative/boundary/errorcode mỗi endpoint. "Test có kỹ không?"

3 thước đo cùng nhau = **bước tranh đầy đủ**: rộng (coverage) + sâu (edge-case) + thật (Recall).

Ví dụ chạy thật: endpoint `bessj`

Đây là "cái bẫy" mà nghiên cứu soi ra — rất dễ kể khi bảo vệ.

SPEC KHAI BÁO

```
GET /api/bessj/{n}/{x}
responses: 200, 401,
403, 404
// KHÔNG có 400!
```

CODE THẬT LÀM

```
if (n < 2 || n > 1000)
    return 400;
// NcsRest.java:43
```

BÃY

```
bessj/1/2.0 → 400
nhưng spec không ghi 400
→ ai chỉ đọc spec
sẽ KHÔNG biết bẫy này
```

→ Test giỏi phải **tự đoán "thử n=1"**. Pilot: AI bắt được bẫy này, một số test người bỏ sót.

 `NcsRest.java:43`

Kết quả Pilot (Tuần 7)

Tín hiệu khả thi — chưa phải kết quả cuối; bản chạy đầy đủ (scale, cùng model) sẽ xác nhận.

100%

RQ1 · coverage (35/35, $p \approx 1.6e-9$)

 [summary.json](#)

217 vs 141

RQ3 · edge-case AI vs Người ($p \approx 6.2e-7$)

 [summary.json](#)

0.135 vs 0.068

RQ2 · Recall EvoMaster > AI = Người

 [summary.json](#)

AI edge-case



217

Người edge-case



141



”Viết ra” ≠ ”Bắt được” — đừng nhầm!

Câu hỏi gài hay gặp: "AI bắt 270 bug à?" → KHÔNG. Đây là 2 thứ khác nhau.

AI VIẾT RA (sản xuất)

295 test case

217 kịch bản edge-case (Người 141)

= số lượng TẠO RA — chưa phải bug.

AI BẮT ĐƯỢC (ground truth)

9/133 lỗi gieo (mutation) = Người

31/31 hành vi lỗi HTTP (4xx/5xx)

32 response 5xx = crash thật.

Vì sao chênh? Test check **status**; mutation đổi **giá trị** → status không đổi → test pass → không bắt.

 [produced-vs-detected.json](#)

ĐỪNG nói: "AI bắt 270 bug" (đó là số test viết ra). **NÓI:** "AI sinh 217 kịch bản; bắt 9 lỗi gieo + kích 31 hành vi lỗi gồm crash 500."

Baseline: ”bản đồ lỗi” trích từ CODE

Trước khi so ai hơn ai, phải biết mỗi endpoint CÓ THỂ lỗi ở đâu — đọc thẳng từ source.

35

endpoint quét

39

điểm-lỗi trong code

22

endpoint có nguy cơ crash 500

5

endpoint có guard 4xx rõ

Tool `error_surface.py` → `results/error-surface-baseline.json` (chạy lại được, không tay).

 `baseline.json`

Đây là baseline GV yêu cầu: trích API + trích code → liệt kê mọi khả năng lỗi per endpoint.

Đọc kết quả thế nào? (trung thực)

Không tô hồng — báo cả chỗ AI thua.

AI thắng ở

Phủ rộng (100% endpoint) + nghĩ ra nhiều kịch bản lỗi/biên hơn người (217 vs 141). AI "chăm" hơn.

AI chưa thắng ở

Bắt lỗi tính toán sâu: EvoMaster ghi lại giá trị đúng làm "đáp án" nên bắt giỏi hơn; AI/người chỉ check status.

Kết luận trung thực: AI **prolific hơn** (sinh nhiều & rộng) nhưng **chưa fault-revealing hơn** — báo đúng, không đổi metric sau khi thấy số.

Điểm yếu & cách tự em xử lý

Tự nêu trước để bảo vệ chủ động.

Chỉ test 1 model (Claude)

Pipeline model-agnostic; kết quả hiện giới hạn ở Claude Sonnet 4.6 — Gemini/GPT để ablation sau (GAP-T).

Chỉ 3 API nhỏ (n=3)

Dùng McNemar gộp theo từng lỗi (N≈133) làm test chính; nêu rõ giới hạn "EMB/JVM-only".

Mutation ≠ lỗi thật 100%

Dùng operator chuẩn PIT/Offutt (có trích dẫn); báo Recall + coverage + edge-case cùng nhau.

Pilot: AI & người cùng 1 agent

Blind protocol (test trước khi gieo lỗi); full run dùng người thật + 1 session Claude Sonnet 4.6 tách biệt.

Chuẩn bị câu hỏi GV hay hỏi

Học thuộc 4 câu này là tự tin bảo vệ.

Hỏi • Điểm mới so với paper cũ là gì?

Đáp • Lần đầu so AI vs người vs EvoMaster trên **lỗi gieo sẵn** → tính được Recall thật (3 cái cũ chưa làm).

Hỏi • Sao chỉ 3 API, có ít quá không?

Đáp • Đây là **pilot** chứng minh khả thi; thống kê chính là McNemar gộp 133 lỗi, không phải $n=3$. Full run mở rộng.

Hỏi • Sao chọn Claude mà không phải GPT?

Đáp • Team chọn Claude Sonnet 4.6: sẵn có + model mạnh non-GPT → thêm bằng chứng ngoài hệ GPT (đóng góp GAP-T). Pipeline model-agnostic nên đổi được.

Hỏi • AI thua EvoMaster ở RQ2 thì sao?

Đáp • Đó là phát hiện trung thực: AI prolific nhưng oracle yếu hơn → đề xuất AI + oracle giá-trị là hướng tương lai.

Phân công & Tiến độ

1 deliverable / người · LR ≠ MS · chia đều.

THÀNH VIÊN	VAI TRÒ	DELIVERABLE
Vo Le Trung Nguyen SE190220	PL	điều phối · merge · nộp · liaison GV
Nguyen Thanh Dat SE190239	DG	dataset + fault catalog + §3
Nguyen Tien Dung SE190034	LR	harness + LLM/EvoMaster + pilot
Nguyen Hoang Huy SE190240	MS	metric + thống kê + §6
Nguyen Le Thuan SE190305	RW	§7 threats + format + slides

Tuần 5–6

Hoàn thiện + nộp proposal · GV duyệt

Tuần 7 — XONG

Pilot: 133 mutant · 3 SUT · AI/Người/EvoMaster

Tuần 8

Full run (scale) Claude Sonnet 4.6 · chốt phân tích + báo cáo

Đóng góp & Bước tiếp

Mới

Lần đầu so **AI vs Người vs EvoMaster** trên benchmark lỗi gieo sẵn → Recall thật.

Khả thi

Pilot: coverage **100%**, edge-case **217>141**, baseline 35 endpoint từ code.

Tiếp

Full run (scale) **Claude Sonnet 4.6** theo protocol đã duyệt → chốt kết luận.

Cảm ơn thầy/cô — nhóm em sẵn sàng nhận câu hỏi. 🙏



Giải ngố — thuật ngữ Research

Slide tra cứu: khi GV hỏi "cái này là gì?" thì mở ra đọc.

REST API — cách phần mềm gọi nhau qua HTTP (GET/POST...).

LLM — mô hình ngôn ngữ lớn (AI như GPT/Claude) đọc & viết text.

GAP — khoảng trống nghiên cứu (cái quan trọng chưa ai làm).

H0 / H1 — giả thuyết "không khác biệt" / "điều muốn chứng minh".

Baseline — mốc/đối thủ để so (Người, EvoMaster).

OpenAPI spec — bản mô tả API (endpoint, input, output) = "hợp đồng".

SLR — đọc tài liệu có hệ thống để tìm khoảng trống.

RQ — câu hỏi nghiên cứu (cái ta đi trả lời).

p-value — xác suất kết quả do may rủi; < 0.05 = đáng tin.

SOTA — state-of-the-art = tốt nhất hiện có.



Giải ngố — thuật ngữ Experiment

Các từ kỹ thuật trong phần chạy thực nghiệm.

Black-box — test chỉ nhìn input/output, KHÔNG nhìn code.

Mutation / mutant — cố ý sửa code tạo "bug giả" để thử test.

Recall — % lỗi gieo mà test bắt được.

Edge-case — kịch bản biên/lỗi (4xx/5xx, giá trị cực trị).

EvoMaster — tool tự sinh test bằng thuật toán tiến hoá.

EP / BVA — chia lớp tương đương / thử giá trị biên (test tay).

Kill (giết mutant) — test pass trên code gốc nhưng fail trên mutant.

Coverage — % endpoint được test chạm tới.

Oracle — "đáp án" test dùng để biết đúng/sai (status / giá trị).

Wilcoxon / McNemar — kiểm định thống kê phi tham số (so cặp).